

Assignment No: 2

Student Details

Name: Tanushka Patil

PRN / Roll No: 123B1B036

Class: BTech

Division: A

Subject: DevOps

Title

Cloud Computing Services – Amazon Web Services (AWS): EC2 Virtual Machine Lifecycle

Aim

To explore the major cloud computing services offered by Amazon Web Services (EC2, S3, Lambda, RDS, Elastic Load Balancing and ECS), and to create, configure, connect to, monitor and terminate a virtual machine instance using AWS EC2.

Theory

Cloud computing is the on-demand delivery of computing resources such as servers, storage, databases, networking and software over the internet with pay-as-you-go pricing. Instead of buying and maintaining physical hardware, users rent capacity from a cloud provider and scale it up or down as needed. Cloud services are commonly grouped into three models: Infrastructure as a Service (IaaS), Platform as a Service (PaaS) and Software as a Service (SaaS).

Amazon Web Services (AWS) is the most widely used public cloud platform. Its core services are:

- **Amazon EC2 (Elastic Compute Cloud):** IaaS service that provides resizable virtual machines called instances. Users choose an AMI (operating system image), an instance type (CPU/RAM), storage (EBS volumes) and networking (VPC, security groups).
- **Amazon S3 (Simple Storage Service):** object storage that stores files as objects inside buckets. It offers 99.999999999% durability, versioning, lifecycle rules and static website hosting.
- **AWS Lambda:** serverless compute service that runs code in response to events (HTTP request, S3 upload, schedule) without provisioning servers. Billing is per request and per millisecond of execution.
- **Amazon RDS (Relational Database Service):** managed relational database service supporting MySQL, PostgreSQL, MariaDB, Oracle and SQL Server. AWS handles backups, patching, replication and failover.
- **Elastic Load Balancing (ELB):** automatically distributes incoming traffic across multiple targets (EC2 instances, containers, IPs). Types include Application Load Balancer (Layer 7), Network Load Balancer (Layer 4) and Gateway Load Balancer.

- **Amazon ECS (Elastic Container Service):** fully managed container orchestration service for running Docker containers, either on EC2 instances or serverless with AWS Fargate.

EC2 Instance Lifecycle

An EC2 instance moves through the states pending → running → stopping → stopped → shutting-down → terminated. A stopped instance keeps its EBS root volume (no compute charge), while a terminated instance is permanently deleted.

Figure: EC2 instance lifecycle



Key EC2 Concepts

- **AMI:** template containing the OS and software used to launch an instance.
- **Instance Type:** hardware configuration, e.g. t2.micro (1 vCPU, 1 GiB RAM, Free Tier eligible).
- **Key Pair:** public/private key pair used for secure SSH login.
- **Security Group:** virtual firewall controlling inbound and outbound traffic to the instance.
- **Amazon CloudWatch:** monitoring service that collects metrics (CPU utilization, network, disk, status checks) and can trigger alarms.

Code / Implementation

Figure: Steps performed in this assignment



Step 1: Sign in and choose a region

- Log in to the AWS Management Console (<https://console.aws.amazon.com>).
- Select a region from the top-right corner, e.g. Asia Pacific (Mumbai) ap-south-1.
- Search for and open the EC2 service.

Step 2: Create (launch) the instance

- Click Launch instance.
- Name: devops-assignment-ec2
- AMI: Amazon Linux 2023 (Free Tier eligible)
- Instance type: t2.micro
- Key pair: Create new key pair → name devops-key, type RSA, format .pem → download it.
- Storage: 8 GiB gp3 (default).

Step 3: Configure network and security group

- Under Network settings, create a security group named devops-sg.
- Inbound rule 1: SSH, TCP port 22, Source: My IP.
- Inbound rule 2: HTTP, TCP port 80, Source: Anywhere (0.0.0.0/0).
- Under Advanced details, add the User data script below to install a web server at launch.
- Click Launch instance and wait until Instance state = Running and Status checks = 2/2 passed.

User data script

```
#!/bin/bash
dnf update -y
dnf install -y httpd
echo "<h1>Hello from AWS EC2 - DevOps Assignment 2</h1>" > /var/www/html/index.html
systemctl enable --now httpd
```

Step 4: Connect to the instance

Option A – EC2 Instance Connect: select the instance → Connect → EC2 Instance Connect → Connect (browser-based terminal).

Option B – SSH from the local machine (Windows PowerShell / Linux terminal):

```
# Restrict key permissions (Linux/macOS)
chmod 400 devops-key.pem

# Connect using the public IPv4 address of the instance
ssh -i "devops-key.pem" ec2-user@<PUBLIC_IP>

# Verify the instance once logged in
whoami
uname -a
cat /etc/os-release
df -h
free -m
sudo systemctl status httpd
```

Step 5: Monitor the instance

- Select the instance → Monitoring tab to view CloudWatch metrics: CPU utilization, Network in/out, Disk reads/writes and Status check failures.
- Open CloudWatch → Alarms → Create alarm → metric EC2 > Per-Instance Metrics > CPUUtilization.
- Condition: Greater than 70% for 5 minutes → notification via an SNS topic (email).
- Generate some load to observe the CPU graph change:

```
sudo dnf install -y stress-ng
stress-ng --cpu 1 --timeout 120s
```

Step 6: Terminate the instance

- Select the instance → Instance state → Terminate (delete) instance → Terminate.

- The state changes to Shutting-down and then Terminated; the instance disappears from the list after about an hour.
- Also delete the security group, key pair and CloudWatch alarm to avoid clutter and charges.

Equivalent AWS CLI commands (optional)

```
aws ec2 run-instances --image-id <AMI_ID> --instance-type t2.micro \
  --key-name devops-key --security-group-ids <SG_ID> \
  --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=devops-assignment-ec2}]"

aws ec2 describe-instances --filters "Name=tag:Name,Values=devops-assignment-ec2" \
  --query "Reservations[.Instances[.InstanceId,State.Name,PublicIpAddress]" --output
table

aws cloudwatch get-metric-statistics --namespace AWS/EC2 --metric-name CPUUtilization \
  --dimensions Name=InstanceId,Value=<INSTANCE_ID> --statistics Average \
  --start-time <START_ISO_TIME> --end-time <END_ISO_TIME> --period 300

aws ec2 terminate-instances --instance-ids <INSTANCE_ID>
```

Output

The expected output at each stage of the EC2 instance lifecycle is shown below.

Expected result in the terminal after connecting:

```
[ec2-user@ip-172-31-10-25 ~]$ whoami
ec2-user
[ec2-user@ip-172-31-10-25 ~]$ curl localhost
<h1>Hello from AWS EC2 - DevOps Assignment 2</h1>
```

Opening `http://<PUBLIC_IP>` in a browser displays the web page served by the instance.

```
$ aws ec2 terminate-instances --instance-ids i-0abc123def4567890
TerminatingInstances:
- CurrentState: {Code: 32, Name: shutting-down}
  InstanceId: i-0abc123def4567890
  PreviousState: {Code: 16, Name: running}
```

Conclusion

In this assignment, I learned about the major AWS cloud services – EC2, S3, Lambda, RDS, ELB and ECS – and their use cases. I launched an EC2 virtual machine by selecting an AMI, instance type, key pair and security group, connected to it securely over SSH, served a web page from it, monitored its performance using CloudWatch metrics and alarms, and finally terminated it to release resources and avoid unnecessary charges.

References

1. Amazon EC2 User Guide – <https://docs.aws.amazon.com/ec2/>

2. Amazon CloudWatch Documentation – <https://docs.aws.amazon.com/cloudwatch/>
3. Overview of Amazon Web Services (Whitepaper) – <https://docs.aws.amazon.com/whitepapers/latest/aws-overview/>
4. AWS CLI EC2 Command Reference – <https://docs.aws.amazon.com/cli/latest/reference/ec2/>

Submission Links

GitHub Repository Link: <https://github.com/tanushkapatil/DevOps>

Hosted / Output Link: http://<EC2_PUBLIC_IP> (instance terminated after completion)